Avaota A1 — BLZ-enabled ZHA & Zigbee2MQTT

Board: Avaota A1 (Ubuntu 24.04 / arm64).

Tip: Before using this guide, flash your ThirdReality Zigbee 3.0 USB dongle (3RZD02032Z) with the latest firmware by following ThirdReality's instructions in the Home Assistant Core BLZ Radios Integration Guide.

After completing step 2 in that guide, plug the ThirdReality zigbee dongle into Avaota A1. You should see it appear as /dev/ttyUSB0. From there, choose either Part A (Run ZHA) or Part B (Run Zigbee2MQTT) for this dongle; do not configure both integrations against the same /dev/ttyUSB0 at the same time.

Part 1 — Download and Flash Official AvaotaOS CLI Image

This step installs the baseline operating system. All subsequent steps assume this official image is used.

1) Go to the AvaotaOS GitHub release page:
https://github.com/AvaotaSBC/AvaotaOS/releases/tag/0.3.0.4

2) Download the following image file:

AvaotaOS-0.3.0.4-noble-cli-arm64-avaota-a1.img.xz

3) Decompress the image if required (balenaEtcher can flash .img.xz directly):

```
xz -d AvaotaOS-0.3.0.4-noble-cli-arm64-avaota-a1.img.xz
```

4) Flash the image to an SD card using balenaEtcher.
5) Insert the SD card into Avaota A1 and power it on.

6) Login with default credentials:
Username: avaota
Password: avaota

Part 2 — Add USB Serial Drivers to Avaota A1

This step is only required if you need to customize the Linux kernel on Avaota A1 to add additional USB–serial drivers for Home Assistant dongles (CH340, CP2102, FTDI, etc.).

2.1 Build Kernel-Only Packages

On an x86_64 Ubuntu build host:

```
git clone https://github.com/AvaotaSBC/AvaotaOS.git
cd AvaotaOS
sudo rm -rf build_dir/
sudo bash build_all.sh \
 -b avaota-a1 \
 -m http://ports.ubuntu.com \
 -v noble \
 -t cli \
 -u avaota \
 -p avaota \
 -s avaota \
 -k yes \
 -g bsp \
 -l no \
 -i no \
 -o yes \
 -e no
```

When the build system drops you into menuconfig, enable the required kernel drivers.

In menuconfig, enable common USB–serial drivers (M or Y):

```
Device Drivers --->
  [*] USB support --->
    <M> USB Serial Converter support
    <M> USB CH341 single port serial driver
    <M> USB FTDI Single Port Serial Driver
    <M> USB CP210x family of UART Bridge Controllers
```

These options ensure Avaota A1 can talk to common Home Assistant Zigbee dongles based on CH340, CP2102, and FTDI USB–UART bridges.

2.2 Kernel Package Output Location

After a successful build, kernel packages are generated under:

```
build_dir/avaota-a1-kernel-pkgs/
```

The following packages are required for kernel update:

- linux-image-*.deb
- linux-dtb-*.deb

Other packages (linux-headers, linux-libc-dev) are optional and not required for runtime.

2.3 Copy Kernel Packages to Avaota A1

Copy the kernel packages to the running Avaota A1 system. Example using SCP:

```
scp -C build_dir/avaota-a1-kernel-pkgs/linux-* \
   avaota@<BOARD_IP>:/tmp/
```

2.4 Install Kernel Packages and Reboot

On the Avaota A1 board:

```
ssh avaota@<BOARD_IP>
cd /tmp

sudo dpkg -i \
 linux-image-*.deb \
 linux-dtb-*.deb

sudo reboot
```

Part 3 — Run Home Assistant

This section switches the system back to cgroup v1 (recommended for Docker on this image), then installs and starts Home Assistant.

3.1 Switch to iptables-legacy

If you encounter nftables / iptables errors, you can switch to iptables-legacy on Avaota A1:

```
sudo update-alternatives --set iptables  /usr/sbin/iptables-legacy
sudo update-alternatives --set ip6tables /usr/sbin/ip6tables-legacy || true
sudo update-alternatives --set arptables /usr/sbin/arptables-legacy || true
sudo update-alternatives --set ebtables  /usr/sbin/ebtables-legacy  || true
```

3.2 Switch Kernel to cgroup v1

Run the following one-line command on Avaota A1 to add cgroup v1 parameters to the kernel command line and reboot:

```
sudo sed -i 's|^\(.*append .*\)$|\1 systemd.unified_cgroup_hierarchy=0
cgroup_enable=cpuset cgroup_enable=memory swapaccount=1|'
/boot/extlinux/extlinux.conf && sudo reboot
```

3.3 Install Docker and Start Home Assistant

After the board has rebooted with cgroup v1:

```
sudo apt-get update
sudo apt-get install -y docker.io containerd
sudo systemctl enable --now docker containerd
```

```
sudo mkdir -p ~/homeassistant/config

sudo docker rm -f homeassistant || true
sudo docker run -d --name homeassistant --restart=unless-stopped \
  --privileged --network=host --uts=host \
  -e TZ=America/Los_Angeles \
  -e PYTHONPATH=/config/deps:/usr/local/lib/python3.13/site-packages \
  -v /run/dbus:/run/dbus:ro \
  -v ~/homeassistant/config:/config \
  ghcr.io/home-assistant/home-assistant:stable
```

Part A — Run ZHA with BLZ

```
cd ~/homeassistant/config

sudo git clone --depth 1 https://github.com/fangzheli/haos_custom_zha_blz.git _tmp_zha
sudo mkdir -p custom_components
sudo mv _tmp_zha/zha custom_components/
sudo rm -rf _tmp_zha

sudo docker exec -it homeassistant bash -lc '
python3 -m pip install --no-deps --force-reinstall \
  --target /config/deps \
  "zha @ git+https://github.com/bouffalolab/zha.git@feat/blz"'
sudo docker restart homeassistant
```

In Home Assistant UI:
Settings → Devices & services → Add Integration → ZHA
Serial device: /dev/ttyUSB0
Baud rate: 2000000

Part B — Run Zigbee2MQTT with BLZ

```
sudo usermod -aG dialout avaota

sudo apt-get install -y mosquitto
sudo systemctl enable --now mosquitto

curl -fsSL https://deb.nodesource.com/setup_lts.x | sudo -E bash -
sudo apt-get install -y nodejs git
sudo corepack enable
sudo corepack prepare pnpm@latest --activate

git clone https://github.com/fangzheli/zigbee-herdsman.git -b feat/blz zigbee-herdsman
git clone https://github.com/fangzheli/zigbee2mqtt.git -b feat/blz-local-dev zigbee2mqtt
```

```
cd ~/zigbee-herdsman
pnpm install
pnpm run build

cd ~/zigbee2mqtt
pnpm install
pnpm run build
cp -n data/configuration.example.yaml data/configuration.yaml

Edit zigbee2mqtt/data/configuration.yaml to at least contain:
homeassistant: true

mqtt:
  server: mqtt://127.0.0.1:1883

serial:
  port: /dev/ttyUSB0
  adapter: blz
  baudrate: 2000000

advanced:
  log_level: debug

cd ~/zigbee2mqtt
nohup pnpm start >/dev/null 2>&1 &

In Home Assistant UI:
Settings → Devices & services → Add Integration → MQTT
Broker: 127.0.0.1
Port: 1883

Zigbee2MQTT will then expose devices and entities via MQTT Discovery.
If you later switch to using ZHA (Part A) with the same dongle, make sure Zigbee2MQTT is
fully stopped so that no other process is holding /dev/ttyUSB0.
```